

Module 7

Real-World Applications

Learning objectives

- Apply UVM to real-world verification scenarios

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["DMA Verification"]

T2["Protocol Verification (Indus)"]

T1 --> T2

T3["Best Practices and Patterns"]

T2 --> T3

Apply UVM to real-world verification scenarios

Overview

- This module applies all learned concepts to real-world verification scenarios. You'll work on compl...

How to learn this module

Suggested learning path

- Module 6 multi-agent protocol env is baseline —
Module 7 applies it to DMA and UART IP blocks
- Review ``module7/dut/dma/simple_dma.v`` register map and descriptor flow before DMA examples
- Study ``module7/dut/protocols/uart.v`` framing (start bit, data, stop) for serial agent labs
- VIP example packages reusable agents — focus on directory layout and naming for reuse
- Best-practices example summarizes production-style TB structure before integrated tests

Design architecture

1. Real-world DUT blocks

- ``module7/dut/dma/simple_dma.v`` —
DMA controller with channel descriptors
- ``module7/dut/protocols/uart.v`` —
serial framing for protocol agent labs
- Blocks represent IP-style interfaces students will see in SoC verification

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
  subgraph soc["module7 IP blocks"]
    DMA[simple_dma]
    UART[uart]
  end
  BUS[internal control] --> DMA
```

2. Verification IP (VIP) architecture

- Reusable agents package protocol knowledge (transactions, sequences, checkers)
- Env composes DMA + UART (or other) agents with shared config and scoreboard fabric
- Best-practices example shows directory layout, naming, and review-friendly TB structure
- VIP boundaries separate protocol expertise from scenario tests

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

VIP[verification IP agents] --> ENV[uvm_env]

ENV --> DMA[(DMA DUT)]

ENV --> UART[(UART DUT)]

ENV --> SB[system scoreboard]

3. Execution pipeline

- Build: Verilator compiles DMA and/or UART RTL; env builds VIP agents and shared infrastructure
- Connect: register bus agent + DMA status monitors + UART analysis paths into system scoreboard
- Sim: software-visible register writes program DMA; UART sequences inject serial traffic concurrently
- Check: DMA transfer completion flags, memory content, UART frame integrity — unified regression rep...

4. Integration view

- Software-visible registers drive DMA; UART provides async serial path
- Scoreboard and coverage span multiple IPs in one regression
- Exercises push toward production-style reuse and documentation
- Module 8 utilities (CLP, recorders) plug into this env style in capstone work

DUT — simple DMA controller

```
/**
 * Module 7: Simple DMA Controller
 *
 * A simple DMA controller for verification.
 *
 * Ports:
 *   clk:          Clock signal
 *   rst_n:        Active-low reset
 *   dma_start:    Start DMA transfer
 *   dma_done:     DMA transfer complete
 *   dma_src_addr: Source address
 *   dma_dst_addr: Destination address
 *   dma_length:   Transfer length
 *   dma_channel:  DMA channel select
 */
```

VIP-style agent reuse

```
"""  
Module 7 Example: VIP (Verification IP) Development  
Demonstrates creating reusable verification IP.  
"""  
  
from pyuvvm import *  
  
# Explicitly import TLM classes that may not be in __all__  
# Try direct imports from known TLM module paths  
try:  
    from pyuvvm.s15_uvm_tlm_1 import uvm_seq_item_pull_port,  
    uvm_analysis_imp  
except (ImportError, AttributeError):  
    try:  
        from pyuvvm.s15_uvm_tlm import uvm_seq_item_pull_port,  
        uvm_analysis_imp  
    except (ImportError, AttributeError):
```

Key files to study

Open these in the repo

- ``module7/dut/dma/simple_dma.v`` — DMA controller with channel descriptors and status
- ``module7/dut/protocols/uart.v`` — UART TX/RX serial interface for protocol agent labs
- ``module7/examples/dma/dma_example.py`` — descriptor programming and transfer completion
- ``module7/examples/protocols/uart_example.py`` — baud/framing stimulus and loopback checks
- ``module7/examples/vip/vip_example.py`` — packaged verification IP agent pattern
- ``scripts/module7.sh`` — ``--dma``, ``--protocols``, ``--vip``, ``--best-practices``, ``--pyuvm-tests``

Verification & testing methods

1. Application-level test methods

- DMA tests: descriptor programming, transfer completion, interrupt/status flags
- UART tests: baud framing, TX/RX loops, error injection in exercises
- VIP tests encapsulate protocol rules so tests focus on scenarios not bit toggling

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

SM[smoke regressions] --> NG[nightly full EXAMPLES]

NG --> REL[release + coverage]

REL --> SHIP[sign-off]

2. Regression and sign-off practices

- ``./scripts/module7.sh`` runs DMA, protocol, VIP, and best-practices demos
- Layered regressions: smoke (short), nightly (full EXAMPLES), release (with coverage goals)
- Waveforms and transaction logs for post-mortem on failures

3. Step-by-step lab execution

- 1. DMA block: ``./scripts/module7.sh --dma`` — program descriptor, confirm completion
- 2. UART protocol: ``./scripts/module7.sh --protocols`` — framing and loopback
- 3. VIP packaging: ``./scripts/module7.sh --vip`` — study reusable agent layout
- 4. Best practices: ``./scripts/module7.sh --best-practices`` — review TB structure checklist
- 5. Integrated: ``./scripts/module7.sh --pyuvm-tests`` — multi-IP regression pass

4. Closure

- ``./scripts/module7.sh --pyuvm-tests``; assessment covers VIP reuse, protocols, and integration
- Prepares for Module 8 utilities used in production regressions (CLP, recorders)
- Articulate which checks live in VIP vs scenario test vs scoreboard

Syllabus topics

1. DMA Verification (1/4)

- DMA Controller Overview
- DMA concepts
- DMA controller architecture
- DMA transfer types
- DMA verification challenges
- DMA Testbench Architecture

1. DMA Verification (2/4)

- Register interface agent
- Memory interface agent
- DMA monitor
- Scoreboard design
- Coverage model
- DMA Verification Scenarios

1. DMA Verification (3/4)

- Simple transfers
- Scatter-gather transfers
- Multiple channel transfers
- Error scenarios
- Performance verification
- DMA Test Implementation

1. DMA Verification (4/4)

- Test scenarios
- Sequence design
- Coverage closure
- Regression testing

2. Protocol Verification (Industry Standards) (1/4)

- UART Verification
- UART protocol
- UART agent design
- UART testbench
- UART verification
- SPI Verification

2. Protocol Verification (Industry Standards) (2/4)

- SPI protocol
- SPI agent design
- Master-slave coordination
- SPI testbench
- I2C Verification
- I2C protocol

2. Protocol Verification (Industry Standards) (3/4)

- I2C agent design
- Multi-master scenarios
- I2C testbench
- AXI Verification
- AXI protocol details
- AXI agent implementation

2. Protocol Verification (Industry Standards) (4/4)

- AXI testbench
- AXI compliance

3. Best Practices and Patterns (1/4)

- Code Organization
- Project structure
- File organization
- Naming conventions
- Documentation standards
- Reusability Patterns

3. Best Practices and Patterns (2/4)

- Component reuse
- Sequence reuse
- Environment reuse
- VIP (Verification IP) creation
- Documentation
- Code documentation

3. Best Practices and Patterns (3/4)

- Test documentation
- User guides
- API documentation
- Maintenance
- Code maintenance
- Test maintenance

3. Best Practices and Patterns (4/4)

- Version management
- Change management

4. Advanced Topics (1/4)

- Performance Optimization
- Testbench optimization
- Simulation speed
- Memory optimization
- CPU utilization
- Coverage Closure

4. Advanced Topics (2/4)

- Coverage strategies
- Coverage analysis
- Coverage improvement
- Coverage metrics
- Regression Testing
- Regression strategies

4. Advanced Topics (3/4)

- Test selection
- Test execution
- Result analysis
- Continuous Integration
- CI/CD setup
- Automated testing

4. Advanced Topics (4/4)

- Result reporting
- Notification systems

5. Verification IP (VIP) Development (1/3)

- VIP Overview
- What is VIP?
- VIP components
- VIP structure
- VIP benefits
- VIP Development

5. Verification IP (VIP) Development (2/3)

- VIP design
- VIP implementation
- VIP testing
- VIP documentation
- VIP Integration
- VIP integration

5. Verification IP (VIP) Development (3/3)

- VIP configuration
- VIP usage
- VIP maintenance

6. System-Level Verification (1/2)

- System Verification
- System architecture
- System testbench
- System scenarios
- System verification
- SoC Verification

6. System-Level Verification (2/2)

- SoC architecture
- SoC testbench
- SoC scenarios
- SoC verification

7. Advanced Debugging (1/2)

- Complex Debugging
- Multi-component debugging
- Transaction flow debugging
- Timing debugging
- Configuration debugging
- Debugging Tools

7. Advanced Debugging (2/2)

- Waveform tools
- Log analysis tools
- Coverage tools
- Performance tools

8. Test Planning and Strategy (1/2)

- Test Planning
- Test strategy
- Test scenarios
- Test coverage
- Test execution plan
- Verification Strategy

8. Test Planning and Strategy (2/2)

- Verification approach
- Verification metrics
- Verification closure
- Sign-off criteria

9. Industry Patterns (1/2)

- Common Patterns
- Industry patterns
- Pattern libraries
- Pattern reuse
- Pattern best practices
- Design Patterns

9. Industry Patterns (2/2)

- Verification patterns
- Architecture patterns
- Implementation patterns
- Testing patterns

10. Project: Build Your Own VIP (1/3)

- Project Requirements
- Choose protocol
- Design VIP
- Implement VIP
- Test VIP
- Document VIP

10. Project: Build Your Own VIP (2/3)

- VIP Components
- Complete agent
- Protocol checker
- Coverage model
- Scoreboard
- Register model (if applicable)

10. Project: Build Your Own VIP (3/3)

- Documentation
- Test suite

Command reference highlights

IP block examples

- ``./scripts/module7.sh --dma`` — DMA descriptor and completion drills
- ``./scripts/module7.sh --protocols`` — UART (and related protocol) agent examples
- ``./scripts/module7.sh --vip --best-practices`` — reusable VIP layout and review-friendly structure

Integrated tests

- ``./scripts/module7.sh --pyuvm-tests`` — integrated pyuvm regressions under ``module7/tests/``
- ``cd module7/tests/pyuvm_tests && make SIM=verilator`` — direct Make when debugging env wiring
- ``./scripts/module7.sh`` — full example sweep plus tests for module sign-off

Layered regression mindset

- Smoke: ``./scripts/module7.sh --dma`` only — fast sanity after tool changes
- Nightly: all example flags — catches cross-IP integration regressions
- Use transaction logs and waves for UART timing vs DMA memory content failures

Hands-on examples

Module 7 orchestrator

```
$ ./scripts/module7.sh --help
```

Terminal: module7_check

```
[[0;36m===== [[0m
[[0;36mModule 7: Real-World Applications [[0m
[[0;36m===== [[0m
```

Usage: ./scripts/module7.sh [OPTIONS]

Module 7: Real-World Applications
This script runs examples and tests for Module 7.

OPTIONS:

Examples:

--dma	Run DMA verification examples
--uart	Run UART protocol examples
--spi	Run SPI protocol examples
--i2c	Run I2C protocol examples
--vip	Run VIP development examples
--best-practices	Run best practices examples
--all-examples	Run all examples (default)
--skip-examples	Skip all examples

Tests:

--pyuvvm-tests	Run pyuvvm tests
----------------	------------------

Environment:

--venv DIR	Virtual environment directory (default: .venv)
--no-venv	Don't use virtual environment
--sim SIMULATOR	Simulator to use (default: verilator)

Other:

--help, -h	Show this help message
------------	------------------------

EXAMPLES:

```
# Run all examples
./scripts/module7.sh
```

Demo: DMA Verification

```
$ .venv/bin/python module7/examples/dma/dma_example.py
```

Terminal: ex01_dma

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module7/examples/dma/dma_
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: UART Protocol

```
$ .venv/bin/python module7/examples/protocols/uart_example.py
```

Terminal: ex02_protocols

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module7/examples/protocol

from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: SPI Protocol

```
$ .venv/bin/python module7/examples/protocols/spi_example.py
```

Terminal: ex03_protocols

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module7/examples/protocol
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: I2C Protocol

```
$ .venv/bin/python module7/examples/protocols/i2c_example.py
```

Terminal: ex04_protocols

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module7/examples/protocol

from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: VIP Development

```
$ .venv/bin/python module7/examples/vip/vip_example.py
```

Terminal: ex05_vip

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module7/examples/vip/vip_
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Best Practices

```
$ .venv/bin/python  
    module7/examples/best_practices/best_practices_example.py
```

Terminal: ex06_best_practices

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module7/examples/best_pra
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Practice & assessment

What you should know (1/3)

- Verify complex designs (DMA, protocols)
- Apply industry best practices
- Create reusable verification IP
- Optimize testbench performance
- Achieve coverage closure
- Plan and execute verification projects

What you should know (2/3)

- Debug complex issues
- Maintain production testbenches
- Apply industry patterns
- Create complete verification solutions
- Register model
- Multiple DMA channels

What you should know (3/3)

- Scatter-gather support
- Performance monitoring
- Coverage model
- Scoreboard

Exercises

- DMA Verification
- Protocol VIP
- Best Practices
- Coverage Closure
- Final Project

Assessment checklist

- Can verify complex designs
- Understands best practices
- Can create reusable VIP
- Can optimize performance
- Can achieve coverage closure
- Can plan verification projects

Summary & next steps

- Apply UVM to real-world verification scenarios
- Complete module7/CHECKLIST.md
- Review module7/EXAMPLES.md and run each lab
- Next: Next module in course